UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Buenos Aires

ALGORITMOS Y ESTRUCTURAS DE DATOS

–2026–

DOCENTE: ING. PABLO MENDEZ

Localización: *Sede Lugano*	**Curso K1153**		
Integrantes del Grupo			
Legajo	**Apellido, Nombre**	**Mail**	**Usuario GitHub**
234.142-6	Rio, Facundo Ariel	frio@frba.utn.edu.ar	FacundoArielRio
233.859-2	Fernandez, Gabriel Omar	gabrfernandez@frba.utn.edu.ar	Gabrfernandez555
218.261-0	Fernández Muruaga, Luciano	lucianofernandez@frba.utn.edu.ar	lucianofernandezmuruaga-utn
208.516-1	Quispe, Juan Manuel	juquispe@frba.utn.edu.ar	juanmaquispe
Fecha de Entrega:	04/11/2026		
Calificación Final:			

Índice

División de Tareas....3
Diagrama de Bloques de Subprogramas....3
Operación 1: Cargar archivo de ataque en memoria....4
Hipótesis....4
Solución Aplicada....4
Casos de Prueba....5
Operación 2: Mostrar ataque cargado en memoria....7
Hipótesis....7
Solución Aplicada....7
Casos de Prueba....11
Operación 3: Crear un archivo de ataque nuevo....15
Hipótesis....15
Solución Aplicada....15
Casos de Prueba....17
Operación 4: Corregir un registro del archivo binario....20
Hipótesis....20
Solución Aplicada....20
Casos de Prueba....24
Operación 5: Corregir un registro en memoria....27
Hipótesis....27
Solución Aplicada....27
Casos de Prueba....29
Operación 6: Guardar memoria en un archivo nuevo....31
Hipótesis....31
Solución Aplicada....31
Casos de Prueba....32
Operación 7: Visualizar ataque cargado en HTML....33
Hipótesis....33
Solución Aplicada....33
Casos de Prueba....34
Problemas Generales....36
Operación 1: Problemas Detectados y Resolución....36
Operación 2: Problemas Detectados y Resolución....36
Operación 3: Problemas Detectados y Resolución....36
Operación 4: Problemas Detectados y Resolución....36

Operación 5: Problemas Detectados y Resolución.....37
Operación 6: Problemas Detectados y Resolución.....37
Operación 7: Problemas Detectados y Resolución.....37
Registro de Herramientas Utilizadas.....38

División de Tareas

[OPERACIÓN 1] → Rio, Facundo Ariel

[OPERACIÓN 2] → Fernández Muruaga, Luciano

[OPERACIÓN 3] → Rio, Facundo Ariel y Fernández Muruaga, Luciano

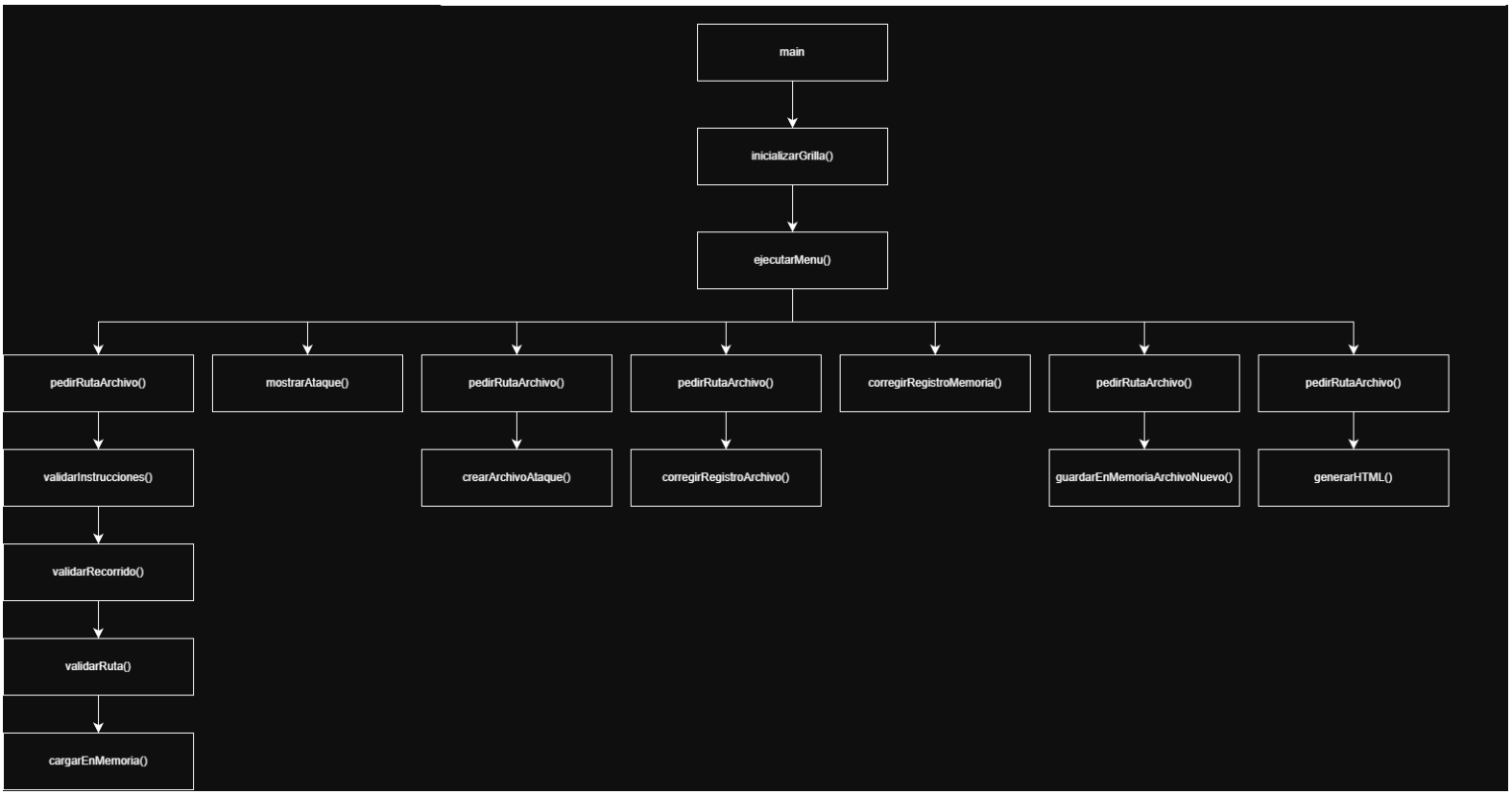
[OPERACIÓN 4] → Quispe, Juan Manuel

[OPERACIÓN 5] → Quispe, Juan Manuel y Fernandez, Gabriel

[OPERACIÓN 6] → Fernandez, Gabriel

[OPERACIÓN 7] → Rio, Facundo Ariel

Diagrama de Bloques de Subprogramas



Operación 1: Cargar archivo de ataque en memoria

Hipótesis

Se parte de la hipótesis de que la matriz estática de 200*200 celdas en RAM debe encontrarse completamente limpia y desocupada antes de procesar una nueva misión. Para lograrlo, los campos de destino deben inicializarse explícitamente en -1 para diferenciar una celda inactiva de una celda activa que apunte a la coordenada (0;0).

Solución Aplicada

El sistema solicita al operador la ruta y nombre del archivo binario y procede a abrirlo en modo de solo lectura binaria, mediante un bucle while se leen secuencialmente los registros de tipo OrdenArchivo desde el disco rígido hacia la memoria RAM y se almacenan en un array temporal buffer[TAM_GRILLA*TAM_GRILLA], contabilizando la cantidad de registros.

Antes de confirmar definitivamente la carga en memoria (validarRuta), el sistema ejecuta de manera secuencial subprogramas de validación para garantizar que el archivo de ataque cumpla con las normativas:

1. validarInstrucciones(...): Recorre el vector utilizando un bucle for para constatar que todas las coordenadas se encuentren estrictamente dentro del rango permitido [0; 199], que exista una única instrucción de despegue (verificando que ésta no posea armamento o ataques asociados), y que exista un único punto de finalización válido (por aterrizaje o ataque kamikaze).
2. validarRecorrido(...): Utiliza una matriz auxiliar de booleanos (visitado[200][200]) para simular el trayecto paso a paso desde el despegue. Audita que no se repitan coordenadas (previniendo ciclos infinitos), valida que los saltos hacia los casilleros siguientes sean estrictamente celdas aledañas (diferencia absoluta en x e y menor o igual a 1, función proporcionada por la librería cmath), y corrobora que no existan registros aislados ajenos al trayecto principal.

Una vez que validarRuta retorna true, se invoca nuevamente a inicializarGrilla() para limpiar la matriz y se procede a mapear cada campo del búfer directamente en la posición cartesiana [x][y] de la matriz en memoria.

Casos de Prueba

```
======================================================
1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo.
5. Corregir un registro de memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.
0. Salir.
======================================================
Ingrese una opcion del 0 al 7: 1


[OPERACION 1] Cargar archivo de ataque en memoria

Ingrese la ruta del archivo:
.
Ingrese el nombre del archivo:
invalido_ciclo
Las coordenadas (0; 0) estan repetidas.


El archivo no cumple las reglas de vuelo. No se cargo en memoria.

======================================================
1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo.
5. Corregir un registro de memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.
0. Salir.
======================================================
Ingrese una opcion del 0 al 7: 1


[OPERACION 1] Cargar archivo de ataque en memoria

Ingrese la ruta del archivo:
.
Ingrese el nombre del archivo:
invalido_despegue_armas
La instrucci n de despegue no puede contener armas ni ataques.


El archivo no cumple las reglas de vuelo. No se cargo en memoria.
```

```
[OPERACION 1] Cargar archivo de ataque en memoria

Ingrese la ruta del archivo:
.
Ingrese el nombre del archivo:
invalido_hueco
Salto no valido hacia (5; 5). No son celdas aledanas.

El archivo no cumple las reglas de vuelo. No se cargo en memoria.
```

```
[OPERACION 1] Cargar archivo de ataque en memoria

Ingrese la ruta del archivo:
.
Ingrese el nombre del archivo:
invalido_sin_despegue
Debe haber una unica instruccion de despegue (encontradas: 0).


El archivo no cumple las reglas de vuelo. No se cargo en memoria.
```

```
[OPERACION 1] Cargar archivo de ataque en memoria

Ingrese la ruta del archivo:
.
Ingrese el nombre del archivo:
invalido_sin_fin
Debe haber una unica instruccion de aterrizaje o ataque kamikaze (encontradas: 0).


El archivo no cumple las reglas de vuelo. No se cargo en memoria.
```

Operación 2: Mostrar ataque cargado en memoria

Hipótesis

La subdivisión del contenido en páginas permitirá al usuario final visualizar el contenido de una forma más simple y controlada; fragmentar el recorrido en un formato de "páginas" es más adecuado para un seguimiento ágil del recorrido.

A partir de esta hipótesis se elaboró la operación planteada, una operación que "*debe mostrar en pantalla el ataque cargado en memoria*".

Solución Aplicada

Para mostrar en pantalla el ataque completo, el primer paso es hallar en qué posición de la tabla está el despegue del drone.

Con ese propósito se optó por un while anidado (un while dentro de otro while), de modo que uno recorra las filas de la tabla y el otro las columnas. Antes de iniciar el ciclo while, se creó una variable booleana llamada *pos_despegue* inicializada como *false*, la cual será la variable de corte del ciclo anidado: una vez se halle la posición de despegue, se detendrán las iteraciones.

```
void mostrarAtaque(Orden grilla[TAM_GRILLA][TAM_GRILLA])
{
    int pos_x = 0, pos_y = 0;
    bool pos_despegue = false;

    while (pos_x < TAM_GRILLA && !pos_despegue) {
        while (pos_y < TAM_GRILLA && !pos_despegue) {
            if (grilla[pos_x][pos_y].despegue == true)
                pos_despegue = true;

            else
                pos_y++;
        }

        if (!pos_despegue) {
            pos_x++;
            pos_y = 0;
        }
    }

    if (!pos_despegue) {
        cout << "No se ha hallado ningun ataque cargado en memoria.\n" << endl;
        return;
    }

    cout << "--- RECORRIDO DEL ATAQUE CARGADO ---\n" << endl;
```

El ciclo anidado recorre, en su while externo, las filas de la tabla; mientras que el interno recorre las columnas de cada fila, por lo que cada iteración revisa una coordenada (*pos_x*, *pos_y*) distinta de la tabla. Para cada posición se utilizó un condicional que evalúa si contiene la instrucción de despegue del drone (ingresando al campo *despegue* del struct), y si es así, la variable de corte pasa a valer *true*, permitiendo salir de los dos while y capturar la *pos_x* y la *pos_y* del despegue.

```
bool pos_despegue = false;

while (pos_x < TAM_GRILLA && !pos_despegue) {
    while (pos_y < TAM_GRILLA && !pos_despegue) {
        if (grilla[pos_x][pos_y].despegue == true)
            pos_despegue = true;
```

El *else* donde se ejecuta *pos_y*++ asegura que se aumente el valor de *pos_y* siempre y cuando el despegue no haya sido hallado en la última celda revisada; mientras que el condicional "if (!pos_despegue)" asegura que luego de completar una fila, se aumente el valor de *pos_x* y se resetee *pos_y* a 0 siempre y cuando el despegue no haya sido hallado en la última columna de la última fila que se revisó.

```
while (pos_x < TAM_GRILLA && !pos_despegue) {
    while (pos_y < TAM_GRILLA && !pos_despegue) {
        if (grilla[pos_x][pos_y].despegue == true)
            pos_despegue = true;
        else
            pos_y++;
    }

    if (!pos_despegue) {
        pos_x++;
        pos_y = 0;
    }
}
```

Asegura que pos_y aumente siempre y cuando el despegue no se haya encontrado en la última coordenada (pos_x, pos_y) revisada

Asegura que pos_x aumente y pos_y se resetee sólo si el despegue no fue encontrado en la última columna de la fila pos_x que se acaba de revisar

Finalmente, el condicional "if (!pos_despegue)" es una capa extra de seguridad que verifica que si no hay ningún despegue en la tabla, entonces no se pase a la sección de mostrar el recorrido del ataque; dado que si no hay despegue, no hay ataque.

```
if (!pos_despegue) {
    cout << "No se ha hallado ningun ataque cargado en memoria.\n" << endl;
    return;
}
```

Si el bloque de ese condicional no se ejecutó, significa que se halló la posición de despegue del drone, y que se puede empezar a mostrar en pantalla el recorrido. Para ello, se optó por incluir un while que verifica que no se haya llegado al fin de ruta, el cual está indicado por una variable booleana llamada *fin_ruta*. Por otra parte, se inicializó una variable *contador_celdas* de tipo int que cuenta la cantidad de celdas que se revisaron.

```
cout << "--- RECORRIDO DEL ATAQUE CARGADO ---\n" << endl;

int contador_celdas = 0;
bool fin_ruta = false;

while (!fin_ruta)
{
    Orden coordenada = grilla[pos_x][pos_y];
```

Dentro del while, se vale de que en el bloque anterior se halló la coordenada (*pos_x*, *pos_y*) del despegue, y se hace un condicional por cada campo de acción del struct: si alguno de esos campos es true, se imprime el texto "-> *[ACCION]*" seguido de la acción contenida en la celda; para el tiempo de espera, se hizo uno condicional que muestra en pantalla el número de segundos que le corresponden a la celda, seguido de la palabra *segundo* si es 1 segundo, y *segundos* para cualquier otra cantidad.

```
while (!fin_ruta)
{
    Orden coordenada = grilla[pos_x][pos_y];

    if (contador_celdas % 5 == 0) {
        cout << "-- PAGINA " << (contador_celdas / 5) + 1 << " --" << endl << endl;
    }

    cout << "- Coordenada: (" << pos_x << "; " << pos_y << ") -" << endl;
    if (coordenada.despegue)
        cout << " -> [ACCION] Despegue" << endl;
    if (coordenada.aterrizaje)
        cout << " -> [ACCION] Aterrizaje" << endl;
    if (coordenada.soltarGranada1)
        cout << " -> [ACCION] Soltar Granada 1" << endl;
    if (coordenada.soltarGranada2)
        cout << " -> [ACCION] Soltar Granada 2" << endl;
    if (coordenada.ataqueKamikaze)
        cout << " -> [ACCION] Ataque Kamikaze" << endl;
    if (coordenada.espera == 1)
        cout << " -> Tiempo de espera: " << coordenada.espera << " segundo" << endl;
    else
        cout << " -> Tiempo de espera: " << coordenada.espera << " segundos" << endl;

    cout << endl;
```

Se utilizó un condicional al final para verificar si la celda revisada contiene el aterrizaje o el ataque kamikaze, lo que cambiaría *fin_ruta* a *true* indicando que concluyó el ataque. Si no, se mueven *pos_x* y *pos_y* a la siguiente coordenada del recorrido, que puede hallarse revisando los campos *siguientex* y *siguientey* del struct. La variable *contador_celdas* se incrementa para indicar que se revisó una nueva celda.

```
        cout << " -> Tiempo de espera: " << coordenada.espera << " segundos" << endl;

    cout << endl;

    if (coordenada.aterrizaje || coordenada.ataqueKamikaze)
    {
        fin_ruta = true;
        cout << "--- FIN DEL ATAQUE ---\n" << endl;
    }
    else
    {
        pos_x = coordenada.siguientex;
        pos_y = coordenada.siguientey;
    }

    contador_celdas++;

    if (contador_celdas % 5 == 0 && !fin_ruta)
    {
        cout << "\n[Presione la tecla 's' para ver la siguiente pagina...]";
        char tecla = getch();
        while (tecla != 's' && tecla != 'S') {
            tecla = getch();
        }
        cout << endl << endl << endl;
    }
```

Se optó por un formato de paginación de celdas de hasta 5 posiciones del recorrido (dado que no se puede saber de antemano la resolución de la pantalla de quien use la función, se utilizó un número pequeño de registros por página). Para ello, cada vez que *contador_celdas* pase por un múltiplo de 5, siempre y cuando no se haya llegado al fin de la ruta, se detendrá la ejecución hasta que el usuario presione la tecla *s* (se incluyó también la *S* mayúscula por si el usuario tiene el Bloq Mayus activado), lo que permitirá pasar a la siguiente página. Por ello, a principio del while se incluyó un bloque que también revisa si *contador_celdas* es un múltiplo de 5 (revisando el resto de su división entera), caso en el cual muestra en pantalla el número de página, que será el resultado de la división entera entre las celdas contadas y 5, sumándole 1 al final para que la primer página sea la página 1 y no la 0 (o dicho de otro modo, comienza en la página 1 y cada vez que *contador_celdas* pase por un múltiplo de 5 se iniciará una página nueva).

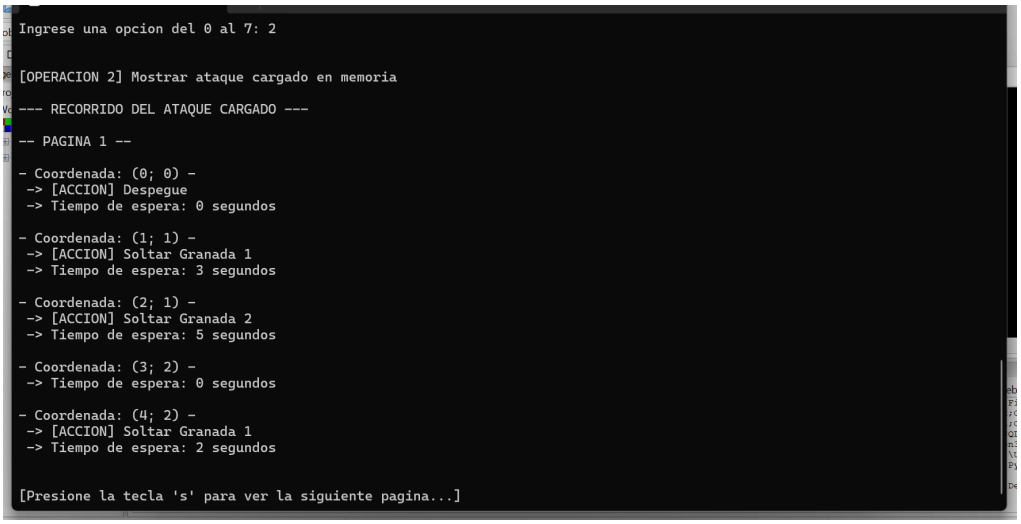
```
while (!fin_ruta)
{
    Orden coordenada = grilla[pos_x][pos_y];

    if (contador_celdas % 5 == 0) {
        cout << "-- PAGINA " << (contador_celdas / 5) + 1 << " --" << endl << endl;
    }
```

Se habían pensando otras opciones para la paginación, por ejemplo tener una variable que cuente la cantidad de líneas mostradas en pantalla (la impresión de la coordenada (*pos_x*, *pos_y*) y la del tiempo de espera de cada celda iban a ser dos líneas, y por cada acción que se muestre en pantalla se sumaría una línea nueva), y se iba a establecer el cambio de página por cantidad de líneas en vez de cantidad de celdas recorridas; lo cual iba a permitir que el cambio de página no dependa tanto de la cantidad de celdas sino de la cantidad de líneas que se imprimieron, de modo que dichas páginas tengan una cantidad de líneas bastante similar sin importar cuántas acciones tuvieron sus celdas; pero eso se descartó porque para el usuario final sería incómodo y poco intuitivo tener una cantidad de posiciones distinta en cada página; es más razonable y da una mayor sensación de control tener una cantidad de celdas uniforme por página.

Casos de Prueba

En caso de tener un archivo de más de 5 registros, se ejecutará cada 5 el getch() que frena la ejecución hasta que el usuario presione *s* para pasar a la siguiente página.



```
Ingrese una opcion del 0 al 7: 2


[OPERACION 2] Mostrar ataque cargado en memoria

--- RECORRIDO DEL ATAQUE CARGADO ---

-- PAGINA 1 --

- Coordenada: (0; 0) -
 -> [ACCION] Despegue
 -> Tiempo de espera: 0 segundos

- Coordenada: (1; 1) -
 -> [ACCION] Soltar Granada 1
 -> Tiempo de espera: 3 segundos

- Coordenada: (2; 1) -
 -> [ACCION] Soltar Granada 2
 -> Tiempo de espera: 5 segundos

- Coordenada: (3; 2) -
 -> Tiempo de espera: 0 segundos

- Coordenada: (4; 2) -
 -> [ACCION] Soltar Granada 1
 -> Tiempo de espera: 2 segundos


[Presione la tecla 's' para ver la siguiente pagina...]
```



```
- Coordenada: (4; 2) -
 -> [ACCION] Soltar Granada 1
 -> Tiempo de espera: 2 segundos


[Presione la tecla 's' para ver la siguiente pagina...]


-- PAGINA 2 --

- Coordenada: (5; 3) -
 -> Tiempo de espera: 0 segundos

- Coordenada: (6; 3) -
 -> [ACCION] Soltar Granada 2
 -> Tiempo de espera: 4 segundos

- Coordenada: (7; 4) -
 -> Tiempo de espera: 0 segundos

- Coordenada: (8; 4) -
 -> [ACCION] Soltar Granada 1
 -> Tiempo de espera: 1 segundo

- Coordenada: (9; 5) -
 -> Tiempo de espera: 0 segundos


[Presione la tecla 's' para ver la siguiente pagina...]
```



```
- Coordenada: (9; 5) -
 -> Tiempo de espera: 0 segundos


[Presione la tecla 's' para ver la siguiente pagina...]


-- PAGINA 3 --

- Coordenada: (10; 5) -
 -> [ACCION] Soltar Granada 2
 -> Tiempo de espera: 3 segundos

- Coordenada: (11; 6) -
 -> Tiempo de espera: 0 segundos

- Coordenada: (12; 6) -
 -> [ACCION] Soltar Granada 1
 -> Tiempo de espera: 2 segundos

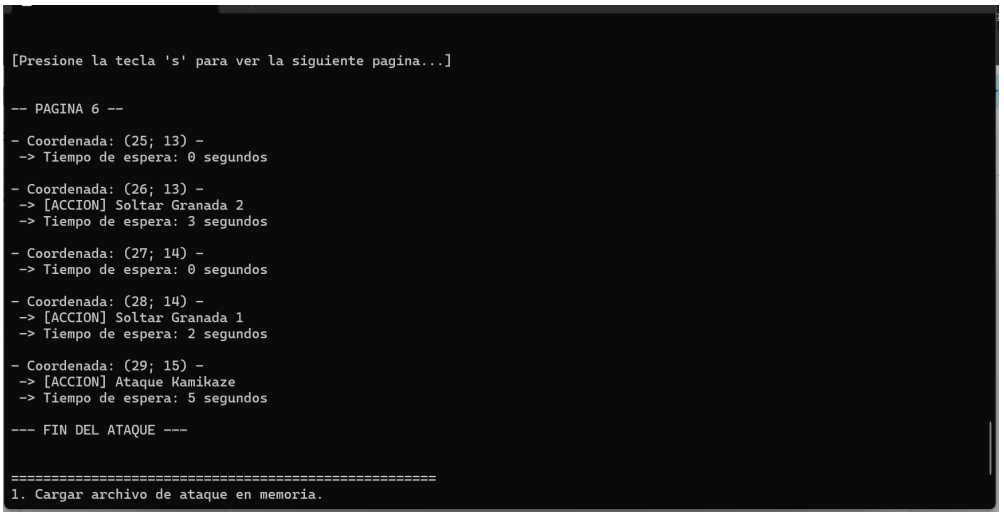
- Coordenada: (13; 7) -
 -> Tiempo de espera: 0 segundos

- Coordenada: (14; 7) -
 -> [ACCION] Soltar Granada 2
 -> Tiempo de espera: 4 segundos


[Presione la tecla 's' para ver la siguiente pagina...]
```







Si el archivo tiene exactamente 5 posiciones en su recorrido, no se ejecuta el getch() para pasar a la siguiente página, dado que se llegó efectivamente al fin de ruta, lo que

hace que no entre en dicho condicional; lo mismo aplica para la última página en caso de tener una cantidad de registros múltiplo de 5.

```
==================================================
Ingrese una opcion del 0 al 7: 2


[OPERACION 2] Mostrar ataque cargado en memoria

--- RECORRIDO DEL ATAQUE CARGADO ---

-- PAGINA 1 --

- Coordenada: (10; 10) -
 -> [ACCION] Despegue
 -> Tiempo de espera: 2 segundos

- Coordenada: (10; 11) -
 -> Tiempo de espera: 3 segundos

- Coordenada: (10; 12) -
 -> [ACCION] Soltar Granada 1
 -> Tiempo de espera: 1 segundo

- Coordenada: (11; 12) -
 -> [ACCION] Soltar Granada 2
 -> Tiempo de espera: 0 segundos

- Coordenada: (12; 12) -
 -> [ACCION] Ataque Kamikaze
 -> Tiempo de espera: 0 segundos

--- FIN DEL ATAQUE ---


==================================================
```

Si el archivo contiene menos de 5 posiciones, simplemente se muestran todos sus registros en la primer página.

```
==================================================
Ingrese una opcion del 0 al 7: 2


[OPERACION 2] Mostrar ataque cargado en memoria

--- RECORRIDO DEL ATAQUE CARGADO ---

-- PAGINA 1 --

- Coordenada: (100; 100) -
 -> [ACCION] Despegue
 -> Tiempo de espera: 0 segundos

- Coordenada: (101; 100) -
 -> [ACCION] Soltar Granada 1
 -> [ACCION] Soltar Granada 2
 -> Tiempo de espera: 0 segundos

- Coordenada: (102; 100) -
 -> [ACCION] Aterrizaje
 -> Tiempo de espera: 0 segundos

--- FIN DEL ATAQUE ---


==================================================
```

Operación 3: Crear un archivo de ataque nuevo

Hipótesis

Se parte de la hipótesis de que la creación de un nuevo plan de vuelo estructurado requiere una interfaz interactiva por consola, donde el usuario defina explícitamente la ruta y el nombre del archivo binario de salida y luego la escritura de múltiples banderas tácticas de forma acumulativa y establecer los tiempos de espera junto con el siguiente destino

Solución Aplicada

El procedimiento de la Operación 3 se compone de dos funciones principales interconectadas:

- **Solicitud de Ubicación de Archivo (pedirRutaArchivo):**

  El sistema invoca una rutina auxiliar que solicita de forma independiente la ruta de directorios y el nombre base del archivo. Mediante el uso de la función de manipulación de cadenas strcat, concatena la ruta, un separador de directorios (/), el nombre ingresado y la extensión obligatoria .dat sobre un búfer seguro (rutaFinal).

- **Apertura del Archivo Binario en Escritura (fopen con "wb"):**

  Se intenta abrir o crear el archivo en disco en modo de escritura binaria. Si ocurre un fallo (por ejemplo, rutas inválidas o permisos denegados), el puntero devuelto es NULL, interrumpiendo el flujo de forma segura para evitar caídas imprevistas del sistema.

- **Captura Interactiva de Registros (OrdenArchivo reg):**

  Mediante un bucle condicionado (do-while), el sistema permite ingresar una cantidad indefinida de registros en base a la respuesta del operador (s o S).

  1. Coordenadas de Posición: Se solicitan y capturan las coordenadas cartesianas (x; y) del nodo actual.
  2. Configuración de Banderas Tácticas por Menú Interno: Se inicializan todas las banderas booleanas en false y se despliega un submenú iterativo (0 al 5) que permite activar de forma granular y acumulativa el despegue, las granadas (1 y 2), el ataque kamikaze o el aterrizaje.
  3. Tiempos de Espera y Destino Subsiguiente: Se capturan los segundos de espera en el casillero y las coordenadas de la siguiente posición (siguientex, siguientey) hacia dónde se desplazará el dron.

- **Persistencia Masiva en Disco (fwrite):**

  Cada registro estructurado completado es volcado de inmediato al archivo binario mediante la función fwrite utilizando el tamaño exacto sizeof(OrdenArchivo), incrementando un contador de registros creados hasta que el usuario decide finalizar la carga y se ejecuta el cierre seguro del archivo (fclose).

Casos de Prueba




C:\Users\facur\OneDrive\Doc
---- Registro 2 ----
Coordenada X entre 0 y 199: 67
Coordenada Y entre 0 y 199: 66
Accion a agregar en (67;66):
0 = Ninguna (terminar actualizacion de acciones del registro)
1 = Despegue (valor actual = 0)
2 = Soltar Granada 1 (valor actual = 0)
3 = Soltar Granada 2 (valor actual = 0)
4 = Ataque Kamikaze (valor actual = 0)
5 = Aterrizaje (valor actual = 0)
Elija una opcion de 0 a 5: 2
Accion a agregar en (67;66):
0 = Ninguna (terminar actualizacion de acciones del registro)
1 = Despegue (valor actual = 0)
2 = Soltar Granada 1 (valor actual = 1)
3 = Soltar Granada 2 (valor actual = 0)
4 = Ataque Kamikaze (valor actual = 0)
5 = Aterrizaje (valor actual = 0)
Elija una opcion de 0 a 5: 3
Accion a agregar en (67;66):
0 = Ninguna (terminar actualizacion de acciones del registro)
1 = Despegue (valor actual = 0)
2 = Soltar Granada 1 (valor actual = 1)
3 = Soltar Granada 2 (valor actual = 1)
4 = Ataque Kamikaze (valor actual = 0)
5 = Aterrizaje (valor actual = 0)
Elija una opcion de 0 a 5: 0
Segundos de espera: 2
Siguiente posicion X entre 0 y 199: 67
Siguiente posicion Y entre 0 y 199: 65
Desea ingresar otro registro? (s o S para si, pulsar otro boton para no):
s

```
---- Registro 3 ----

Coordenada X entre 0 y 199: 67
Coordenada Y entre 0 y 199: 65

Accion a agregar en (67;65):

  0 = Ninguna (terminar actualizacion de acciones del registro)
  1 = Despegue (valor actual = 0)
  2 = Soltar Granada 1 (valor actual = 0)
  3 = Soltar Granada 2 (valor actual = 0)
  4 = Ataque Kamikaze (valor actual = 0)
  5 = Aterrizaje (valor actual = 0)

Elija una opcion de 0 a 5: 4

Accion a agregar en (67;65):

  0 = Ninguna (terminar actualizacion de acciones del registro)
  1 = Despegue (valor actual = 0)
  2 = Soltar Granada 1 (valor actual = 0)
  3 = Soltar Granada 2 (valor actual = 0)
  4 = Ataque Kamikaze (valor actual = 1)
  5 = Aterrizaje (valor actual = 0)

Elija una opcion de 0 a 5: 0

Segundos de espera: 2

Siguiente posicion X entre 0 y 199: 67
Siguiente posicion Y entre 0 y 199: 64

Desea ingresar otro registro? (s o S para si, pulsar otro boton para no):
n
```

Operación 4: Corregir un registro del archivo binario

Hipótesis

Se parte de la hipótesis de que es necesario modificar o corregir de forma directa las instrucciones tácticas de un nodo específico de vuelo previamente guardado en un archivo binario .dat en el disco rígido, sin necesidad de cargar toda la grilla en la memoria RAM ni rehacer el archivo completo.

Asimismo, se asume que el sistema debe abrir el archivo en modo de lectura/escritura aleatoria, localizar secuencialmente la posición física del registro correspondiente a la coordenada (x;y) indicada por el operador, capturar los nuevos datos y sobrescribir de forma exacta ese bloque de memoria en el disco mediante acceso directo con fseek.

Solución Aplicada

Para desarrollarla se implementó la función corregirRegistroArchivo(), ubicada en archivos.cpp. Se tomó como referencia el ejercicio de modificación de archivos binarios trabajado previamente en la cursada, utilizando principalmente las funciones fopen, fseek, ftell, fread y fwrite.

El procedimiento se dividió en:

- **Solicitud y apertura del archivo:** primero se utiliza la función pedirRutaArchivo() para pedirle al usuario la ubicación y el nombre del archivo que quiere modificar. Luego mediante fopen con el modo “rb+” se abre el archivo para leer y escribir sin eliminar los registros que ya contiene. Si el mismo no puede abrirse, se muestra un mensaje de error y finaliza la función.

```
void corregirRegistroArchivo()
{
    char rutaArchivo[384];
    pedirRutaArchivo(rutaArchivo);

    FILE *f = fopen(rutaArchivo, "rb+");
    if (f == NULL)
    {
        cout << "\nNo se pudo abrir el archivo: " << rutaArchivo << endl;
        return;
    }
```

- **Ingreso y validación de las coordenadas:** se solicita al usuario las coordenadas X e Y que quiere corregir. Mediante un if se verifica que ambas estén dentro del rango permitido.

```
int x, y;
cout << "\nIngrese la coordenada X del registro a corregir: ";
cin >> x;
cout << "Ingrese la coordenada Y del registro a corregir: ";
cin >> y;

if (x < 0 || x >= TAM_GRILLA || y < 0 || y >= TAM_GRILLA)
{
    cout << "\nCoordenadas incorrectas" << endl;
    fclose(f);
    return;
}
```

- **Cálculo y búsqueda del registro:** en este procedimiento es donde aparece la implementación de una parte del ejercicio 2 de la guía de "Archivos", clave fundamental para el desarrollo de la Operación 4. Para saber cuántos registros contiene el archivo se utiliza fseek para posicionarse en el final y luego ftell para obtener su tamaño. Ese valor lo dividimos por sizeof(OrdenArchivo) y así obtenemos la cantidad de registros almacenados. Una vez que volvemos al comienzo del archivo, se utiliza un ciclo while junto a un fread para leer los registros uno por uno. En cada ciclo se comparan las coordenadas del registro con las ingresadas por el usuario. Si coinciden se guarda esa posición en la variable posicion y se detiene la búsqueda. En caso de no encontrarlo, se informa al usuario.

```
fseek(f, 0, SEEK_END);
int tam = ftell(f);
int cantidad = tam / sizeof(OrdenArchivo);
fseek(f, 0, SEEK_SET);

OrdenArchivo reg;
int posicion = -1;

int i = 0;
while (i < cantidad && posicion == -1)
{
    fread(&reg, sizeof(OrdenArchivo), 1, f);

    if (reg.x == x && reg.y == y)
        posicion = i;

    i++;
}

if (posicion == -1)
{
    cout << "\nNo existe un registro con esas coordenadas." << endl;
    fclose(f);
    return;
}
```

Operación 4 - TP Algoritmos

```
int modifica(char ruta[])
{
    int tam, cantidad,n, aux;
    FILE *f;
    if (f = fopen(ruta,"rb+"))
    {
        cout << "Ingrese el n que quiere modificar" << endl;
        cin >> n;

        fseek(f,0, SEEK_END);
        tam = ftell(f);
        cantidad = tam /sizeof(int);

        if (n>cantidad)
            cout << "Error, no existe n" << endl;
        else
        {
            cout << "Ingrese el nuevo valor" << endl;
            cin >> aux;

            fseek(f,sizeof(int)*(n-1), SEEK_SET);
            fwrite(&aux,sizeof(int),1,f);

        }
        fclose(f);
        return 0;
    }
    else
        return 1;
}
```

Ejercicio 2 - Guía "Archivos"
Código que fue adaptado
a la Operación 4

- **Modificación de los registros:** una vez que el registro es encontrado, se muestran en pantalla todos los datos almacenados. Luego se crea una nueva variable de tipo OrdenArchivo, copiando inicialmente los datos del registro.

```
cout << "\n--- REGISTRO ENCONTRADO ---" << endl;
cout << "X: " << reg.x << endl;
cout << "Y: " << reg.y << endl;
cout << "Espera: " << reg.espera << endl;
cout << "Soltar granada 1: " << reg.soltarGranada1 << endl;
cout << "Soltar granada 2: " << reg.soltarGranada2 << endl;
cout << "Ataque kamikaze: " << reg.ataqueKamikaze << endl;
cout << "Aterrizaje: " << reg.aterrizaje << endl;
cout << "Despegue: " << reg.despegue << endl;
cout << "Siguiente X: " << reg.siguientex << endl;
cout << "Siguiente Y: " << reg.siguientey << endl;

OrdenArchivo nuevo = reg;
```

El usuario ingresa los valores que desea modificar y mantiene iguales los que no. Antes de guardar, se comprueba que las coordenadas ingresadas estén dentro de los límites de la grilla.

```
cout << "\nIngrese los nuevos datos del registro:" << endl;
cout << "Nueva X (0 a 199): ";
cin >> nuevo.x;
cout << "Nueva Y (0 a 199): ";
cin >> nuevo.y;
cout << "Nueva espera (segundos): ";
cin >> nuevo.espera;
cout << "Soltar granada 1 (1=Si, 0=No): ";
cin >> nuevo.soltarGranada1;
cout << "Soltar granada 2 (1=Si, 0=No): ";
cin >> nuevo.soltarGranada2;
cout << "Ataque kamikaze (1=Si, 0=No): ";
cin >> nuevo.ataqueKamikaze;
cout << "Aterrizaje (1=Si, 0=No): ";
cin >> nuevo.aterrizaje;
cout << "Despegue (1=Si, 0=No): ";
cin >> nuevo.despegue;
cout << "Siguiente X (0 a 199): ";
cin >> nuevo.siguientex;
cout << "Siguiente Y (0 a 199): ";
cin >> nuevo.siguientey;
```

```
if (nuevo.x < 0 || nuevo.x >= TAM_GRILLA ||
    nuevo.y < 0 || nuevo.y >= TAM_GRILLA ||
    nuevo.siguientex < 0 || nuevo.siguientex >= TAM_GRILLA ||
    nuevo.siguientey < 0 || nuevo.siguientey >= TAM_GRILLA)
{
    cout << "\nDatos incorrectos. No se modifico el archivo." << endl;
    fclose(f);
    return;
}
```

- **Registro del registro en el archivo:** finalmente se utiliza nuevamente fseek, pero esta vez para posicionarse donde se encuentra el registro que queremos modificar. Para eso se multiplica la variable posicion por sizeof(OrdenArchivo). Una vez ubicado, se usa un fwrite que sobrescribe el registro anterior (reg) con los nuevos datos (nuevo). Por último se cierra el archivo con fclose(f).

```
fseek(f,posicion * sizeof(OrdenArchivo),SEEK_SET);
fwrite(&nuevo,sizeof(OrdenArchivo),1,f);
cout << "\nRegistro corregido y guardado en el archivo." << endl;

fclose(f);
```

Código del Ejercicio 2 - Guía "Archivos" ya aplicado en el final de la Operación 4

Casos de Prueba

```
[OPERACION 2] Mostrar ataque cargado en memoria

--- RECORRIDO DEL ATAQUE CARGADO ---

-- PAGINA 1 --

- Coordenada: (2; 2) -
 -> [ACCION] Despegue
 -> Tiempo de espera: 5 segundos

- Coordenada: (3; 3) -
 -> Tiempo de espera: 0 segundos

- Coordenada: (2; 4) -
 -> Tiempo de espera: 0 segundos

- Coordenada: (3; 5) -
 -> [ACCION] Soltar Granada 1
 -> Tiempo de espera: 0 segundos

- Coordenada: (4; 5) -
 -> Tiempo de espera: 5 segundos


[Presione la tecla 's' para ver la siguiente pagina...]


-- PAGINA 2 --

- Coordenada: (5; 5) -
 -> [ACCION] Aterrizaje
 -> Tiempo de espera: 0 segundos

--- FIN DEL ATAQUE ---
```

Registro guardado exitosamente

```
=====================================================
1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo.
5. Corregir un registro de memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.
0. Salir.
=====================================================
Ingrese una opcion del 0 al 7: 4


[OPERACION 4] Corregir un registro del archivo binario

Ingrese la ruta del archivo:
C:\Users\jmqqu\OneDrive\Escritorio\TP-Algoritmos-Drones-main\TP-Algoritmos-Drones-main\TP-Algoritmos
Ingrese el nombre del archivo:
guerra

Ingrese la coordenada X del registro a corregir: 4
Ingrese la coordenada Y del registro a corregir: 5
```

```
--- REGISTRO ENCONTRADO ---
X: 4
Y: 5
Espera: 0
Soltar granada 1: 0
Soltar granada 2: 0
Ataque kamikaze: 0
Aterrizaje: 0
Despegue: 0
Siguiente X: 5
Siguiente Y: 5

Ingrese los nuevos datos del registro:
Nueva X (0 a 199): 4
Nueva Y (0 a 199): 5
Nueva espera (segundos): 0
Soltar granada 1 (1=Si, 0=No): 0
Soltar granada 2 (1=Si, 0=No): 1
Ataque kamikaze (1=Si, 0=No): 0
Aterrizaje (1=Si, 0=No): 0
Despegue (1=Si, 0=No): 0
Siguiente X (0 a 199): 5
Siguiente Y (0 a 199): 5

Registro corregido y guardado en el archivo.
```

No existe la coordenada

```
[OPERACION 4] Corregir un registro del archivo binario

Ingrese la ruta del archivo:
C:\Users\jmqqu\OneDrive\Escritorio\TP-Algoritmos-Drones-main\TP-Algoritmos-Drones-main\TP-Algoritmos
Ingrese el nombre del archivo:
guerra

Ingrese la coordenada X del registro a corregir: 1
Ingrese la coordenada Y del registro a corregir: 1

No existe un registro con esas coordenadas.
```

No se modifica el archivo

```
[OPERACION 4] Corregir un registro del archivo binario

Ingrese la ruta del archivo:
C:\Users\jmqqu\OneDrive\Escritorio\TP-Algoritmos-Drones-main\TP-Algoritmos-Drones-main\TP-Algoritmos
Ingrese el nombre del archivo:
guerra

Ingrese la coordenada X del registro a corregir: 4
Ingrese la coordenada Y del registro a corregir: 5

--- REGISTRO ENCONTRADO ---
X: 4
Y: 5
Espera: 5
Soltar granada 1: 0
Soltar granada 2: 0
Ataque kamikaze: 0
Aterrizaje: 0
Despegue: 0
Siguiente X: 5
Siguiente Y: 5

Ingrese los nuevos datos del registro:
Nueva X (0 a 199): 200
Nueva Y (0 a 199): 200
Nueva espera (segundos): 0
Soltar granada 1 (1=Si, 0=No): 0
Soltar granada 2 (1=Si, 0=No): 0
Ataque kamikaze (1=Si, 0=No): 0
Aterrizaje (1=Si, 0=No): 0
Despegue (1=Si, 0=No): 0
Siguiente X (0 a 199):
5
Siguiente Y (0 a 199): 5

Datos incorrectos. No se modifico el archivo.
```

Operación 5: Corregir un registro en memoria

Hipótesis

Se parte de la hipótesis de que las modificaciones requeridas sobre una misión ya cargada deben poder realizarse de manera directa y dinámica en la matriz de la memoria RAM (200*200), evitando la necesidad de reescribir todo el archivo binario en disco para cambios menores.

Asimismo, se asume que antes de acceder a la grilla, el sistema debe validar estrictamente las coordenadas cartesianas ingresadas por el operador para prevenir desbordamientos espaciales o fallas de segmentación en la memoria del programa.

Solución Aplicada

El procedimiento modular de la Operación 5 implementado en la función corregirRegistroMemoria se compone de las siguientes etapas lógicas:

- **Ingreso y Validación de Fronteras:**

  El sistema solicita al operador las coordenadas espaciales (x;y) a corregir dentro del rango permitido [0, 199]. Mediante una estructura condicional de validación de límites, se comprueba si las coordenadas se encuentran fuera de la grilla; de ser así, se emite un mensaje de error y se interrumpe la ejecución de forma segura.

- **Consulta e Impresión del Estado Actual:**

  Utilizando una referencia directa a la estructura alojada en la grilla (Orden &celda = grilla[x][y]), el programa recupera y muestra por consola el estado actual de la casilla seleccionada (indicando banderas activas de despegue, aterrizaje, granadas, ataque kamikaze, segundos de espera y la posición de la celda siguiente), permitiendo al operador visualizar qué parámetros va a modificar.

- **Actualización Interactiva de Banderas Tácticas:**

  A través de un bucle iterativo controlado por un menú numérico (0 al 5), el operador puede activar de forma acumulativa y granular las opciones tácticas deseadas para esa posición específica, evaluando el estado actual de cada indicador en tiempo real hasta pulsar 0 para finalizar la actualización de acciones.

- **Modificación de Tiempos y Enlaces de Destino:**

Se capturan de forma directa los nuevos valores correspondientes a los segundos de espera y las coordenadas subsiguientes de la trayectoria (siguientex y siguientey). Adicionalmente, el sistema evalúa que las coordenadas de destino ingresadas no rebasen los límites del tablero, emitiendo una advertencia en caso de registrarse valores fuera de rango. Finalmente, se confirma la actualización exitosa de la estructura en la RAM.

Casos de Prueba



```
Accion a agregar en (14;14):

  0 = Ninguna (terminar actualizacion de acciones del registro)
  1 = Despegue (valor actual = 0)
  2 = Soltar Granada 1 (valor actual = 0)
  3 = Soltar Granada 2 (valor actual = 0)
  4 = Ataque Kamikaze (valor actual = 1)
  5 = Aterrizaje (valor actual = 1)

Elija una opcion de 0 a 5: 0
Segundos de espera: 15
Siguiente posicion X (0 a 199): 14
Siguiente posicion Y (0 a 199): 13

El registro en memoria RAM (14; 14) fue actualizado correctamente.
```

Operación 6: Guardar memoria en un archivo nuevo

Hipótesis

Se parte de la hipótesis de que se requiere guardar el estado actual de la grilla en RAM en un archivo binario nuevo, utilizando una ruta y nombre definidos por el usuario. Dado que la matriz bidimensional es de 200*200 (40.000 celdas) y contiene en su mayoría casilleros vacíos, el sistema debe recorrer la memoria y filtrar únicamente los casilleros que contengan registros con acciones, tiempos de espera o conexiones de ruta activas para grabarlos en disco de forma compacta.

Solución Aplicada

El procedimiento de la operación 6 consta de dos funciones principales:

- Solicitud de Ubicación de Archivo (pedirRutaArchivo(rutaNuevo))

Se invoca la rutina ya mencionada anteriormente para que el usuario ingrese la nueva ruta para el archivo que se va a crear.

- Escritura Filtrada desde la Memoria a Disco (guardarMemoriaArchivoNuevo):
    1. Apertura en modo escritura binaria ("wb"): Se utiliza fopen con el modo "wb". Se valida que el puntero no sea NULL; en caso de error de acceso o ruta, se notifica y se interrumpe la función.
    2. Recorrido y Filtrado: Se realiza un doble bucle for para recorrer las 40.000 celdas de la grilla cartesiana en RAM (i;j). Se evalúa una condición lógica combinada (if) que verifica si el casillero tiene banderas activas (despegue, aterrizaje, granadas, kamikaze), tiempo de espera espera > 0, o un puntero de destino válido (siguientex, siguientey).
    3. Mapeo a OrdenArchivo y Escritura en Disco (fwrite): Por cada celda que cumple la condición, se transfiere la información al registro de persistencia OrdenArchivo reg fijando reg.x = i y reg.y = j, y se graba directamente en el flujo binario usando fwrite(&reg, sizeof(OrdenArchivo), 1, f).
    4. Conteo y Cierre: Se incrementa la variable registrosGuardados para registrar cuántas estructuras fueron consolidadas en el almacenamiento secundario, finalizando con la liberación del archivo mediante fclose(f) y el mensaje de confirmación por consola.

Casos de Prueba

```
C:\Users\facur\OneDrive\Doc

=====================================================
1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo.
5. Corregir un registro de memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.
0. Salir.
=====================================================
Ingrese una opcion del 0 al 7: 1


[OPERACION 1] Cargar archivo de ataque en memoria

Ingrese la ruta del archivo:
.
Ingrese el nombre del archivo:
valido_diagonal

Archivo cargado en memoria exitosamente. Cantidad de registros: 15

=====================================================
1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo
4. Corregir un registro del archivo.
5. Corregir un registro de memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.
0. Salir.
=====================================================
Ingrese una opcion del 0 al 7: 6


[OPERACION 6] Guardar memoria en un archivo nuevo

Ingrese la ruta del archivo:
.
Ingrese el nombre del archivo:
valido_diagonal
Memoria guardada exitosamente en el nuevo archivo. Total de registros: 15
```

Operación 7: Visualizar ataque cargado en HTML

Hipótesis

Se parte de la hipótesis de que la validación y supervisión de una misión de drones de 200*200 no puede limitarse exclusivamente a una salida por consola, ya que esto dificulta la comprensión del recorrido.

Por lo tanto, se asume que el sistema debe ser capaz de exportar de manera autónoma el estado actual de la grilla en la memoria hacia un documento en formato HTML incorporando un diseño visual adaptativo.

Solución Aplicada

El procedimiento de la Operación 7 se compone de un flujo estructurado entre el bloque de opciones del menú principal y el subsistema de visualización (visualizador.h / visualizador.cpp):

- Adecuación y Cambio Automático de Extensión:

  El sistema solicita la ruta del archivo. Al ingresar automáticamente la extensión .dat, el programa la detecta mediante strlen y strcmp, reemplazándola por la extensión .html (strcpy) para garantizar que el reporte web se genere correctamente.

- Apertura de Flujo en Modo Texto (fopen con "w"):

  Se abre el archivo de destino en disco en modo de escritura de texto. Si ocurre un fallo en la creación, el puntero devuelto es NULL, informando el error por consola y abortando de forma segura.

- Inyección de Estilos CSS y Estructura Base:

  Se utiliza la función fprintf para estructurar el documento HTML, utilizando de forma directa una constante de estilos (CSS_ESTILOS) donde está la personalización del html

- Recorrido Bidireccional de la Matriz RAM (200 * 200):

  Mediante un bucle anidado que recorre los 40.000 casilleros de la grilla, el sistema evalúa el estado operativo de cada celda y escribe de forma condicional etiquetas <td> dotadas de clases estilizadas específicas y atributos flotantes dinámicos:

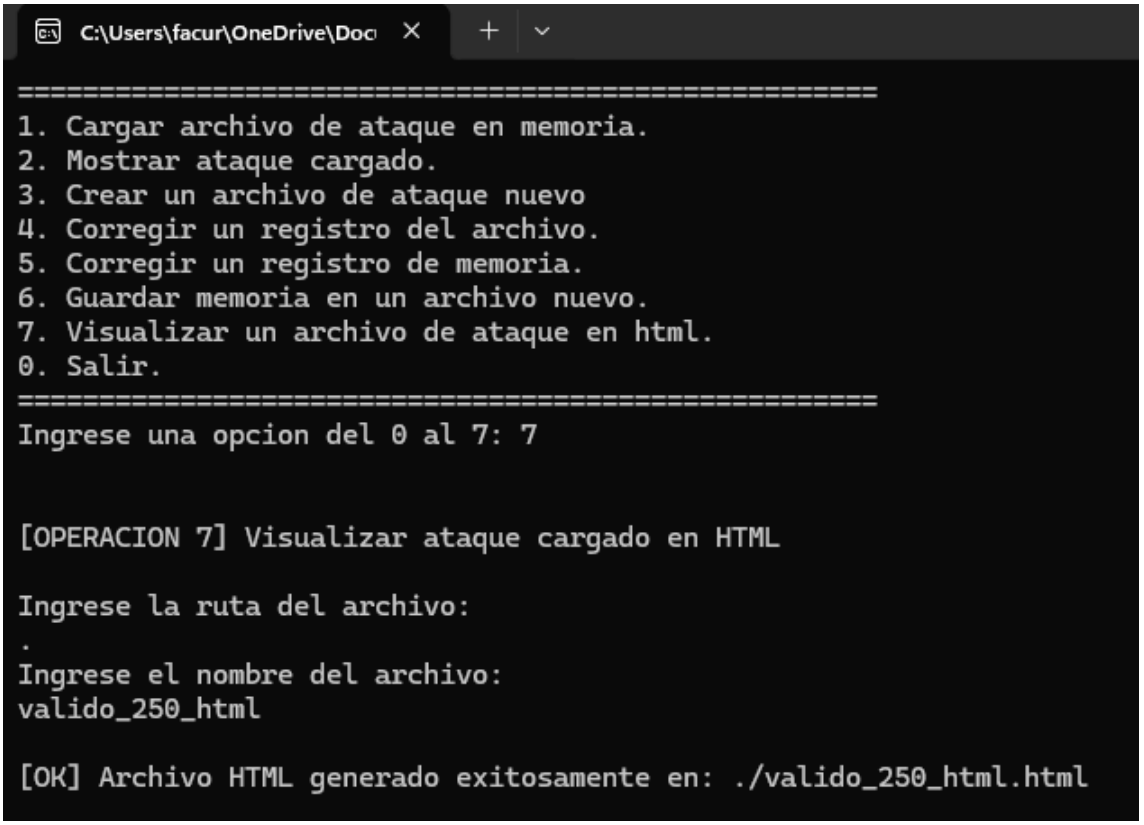
  1. Despegue: Identificado con la clase .c-despegue (color verde), mostrando un título interactivo Despegue (x;y).

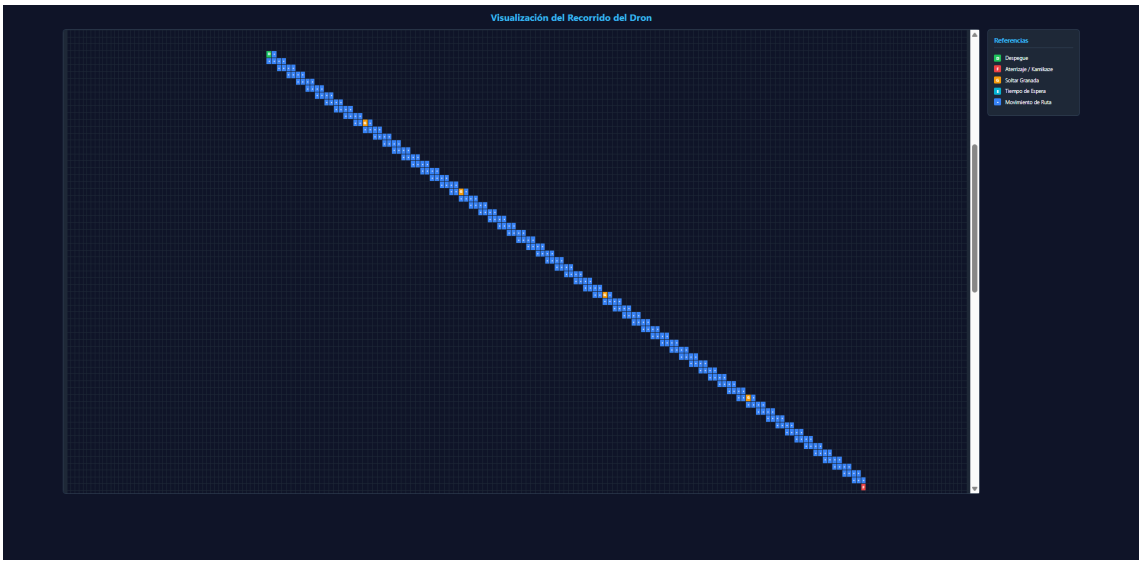
2. Aterrizaje o Ataque Kamikaze: Identificado con la clase .c-kamikaze (color rojo), mostrando el indicador de fin de misión Fin (x;y).
3. Lanzamiento de Granadas: Identificado con la clase .c-granada (color ámbar/naranja), mostrando Granada (x;y).
4. Tiempos de Espera: Identificado con la clase .c-espera (color cian), reflejando los segundos específicos de demora Espera Xs (x;y).
5. Tramos de Ruta Válidos: Identificado con la clase .c-paso (color azul con un punto central •), mostrando Paso (x;y).
6. Celdas Inactivas: Casilleros vacíos estilizados con un borde sutil para estructurar la grilla cartesiana completa sin ensuciar la interfaz, incluyendo la coordenada respectiva en el atributo title="Vacio (x;y)".

- Sidebar y Cierre de Archivo:

  Finalmente, se inserta un panel lateral de referencias tácticas (.sidebar) con la leyenda explicativa de los códigos de colores y símbolos, cerrando el archivo de manera segura con fclose.

Casos de Prueba




Visualización del Recorrido del Dron
Referencias
Despegue
Aterrizaje / Kamikaze
Soltar Granada
Tiempo de Espera
Movimiento de Ruta

Problemas Generales

Operación 1: Problemas Detectados y Resolución

En las etapas iniciales de desarrollo, al intentar cargar archivos binarios generados con versiones preliminares del código, las celdas inactivas de la matriz contenían basura en memoria o ceros por defecto. Como la coordenada (0,0) es un casillero válido, el sistema interpretaba erróneamente que existían conexiones fantasma, generando falsos positivos o bloqueos imprevistos en el motor de validación de rutas.

Se implementó una rutina de inicialización obligatoria al arrancar el programa que asigna el valor -1 a los destinos inactivos, y se descartaron los archivos de prueba obsoletos.

Operación 2: Problemas Detectados y Resolución

Durante el desarrollo de la función mostrarAtaque, tuvimos algunas dudas sobre la forma correcta de pasar la matriz de la grilla como parámetro utilizando la constante TAM_GRILLA proveniente de *drone.h*; habíamos considerado usar (grilla[][TAM_GRILLA], int TAM_GRILLA) o (grilla[][TAM_GRILLA], int filas=TAM_GRILLA). Tras consultar la sintaxis adecuada para asegurar que el compilador reconociera las dimensiones de la estructura, optamos por definir los corchetes utilizando directamente la constante (grilla[TAM_GRILLA][TAM_GRILLA]), logrando así que el tamaño de las filas y columnas quedara correctamente especificado de manera uniforme en el subprograma.

Operación 3: Problemas Detectados y Resolución

En el diseño preliminar, al estructurar el bucle de acciones tácticas, las opciones ingresadas por el usuario sobreescribían o reseteaban los valores booleanos previos de forma involuntaria si se intentaba seleccionar más de una condición o si se ingresaban opciones fuera de rango (como letras o números mayores a 5).

Se diseñó un bucle interno independiente con validación por switch-case que muestra en tiempo real el valor actual de cada bandera (valor actual = 0 o 1), permitiendo activar selectivamente las opciones deseadas sin perder la consistencia del registro antes de grabarlo de manera definitiva en el archivo binario.

Operación 4: Problemas Detectados y Resolución

En las fases iniciales de desarrollo, se cometían dos errores frecuentes al intentar modificar un archivo en disco: en primer lugar, se abría el archivo en modo de sobrescritura total ("wb"), lo que eliminaba todo el contenido guardado previamente en lugar de editarlo; en segundo lugar, tras realizar la búsqueda con fread, el puntero

interno de lectura del archivo quedaba avanzado un registro por delante, provocando que la posterior escritura con fwrite modificara la casilla

Se solucionó implementando la apertura en modo binario de actualización "rb+" y haciendo uso explícito del cálculo de desplazamientos absolutos mediante fseek(f, posicion * sizeof(OrdenArchivo), SEEK_SET) justo antes de invocar a fwrite. Esto garantizó situar el puntero de escritura en la posición física exacta del registro deseado dentro del archivo .dat sin corromper ni desalinear el resto de las órdenes cargadas.

Operación 5: Problemas Detectados y Resolución

Al igual que en las primeras versiones de la Operación 5 se omitió la posibilidad de actualizar múltiples estados operativos de forma simultánea sobre un casillero existente en la grilla. Modificar una celda obligaba a redefinir toda la estructura o pisaba accidentalmente las acciones previas que ya se encontraban cargadas en la RAM.

Se implementó la misma lógica acumulativa de actualización por menú iterativo (0 al 5) utilizando referencias directas a la celda en la matriz (Orden &celda = grilla[x][y]). Gracias a esto, el operador puede consultar el estado actual de la celda ubicada en (x;y) e ir sumando o modificando banderas tácticas, tiempos de espera y coordenadas subsiguientes de manera flexible antes de actualizar la estructura en memoria.

Operación 6: Problemas Detectados y Resolución

Durante el desarrollo, uno de los inconvenientes más complejos estuvo relacionado directamente con la condición lógica (if) dentro del doble bucle de recorrido. En las versiones preliminares, el filtro evaluaba únicamente si existían acciones directas o si espera > 0, pero omitía la validación integral de los punteros siguientex e siguientey junto con las posiciones de la grilla i y j. Como consecuencia directa de esto, el sistema no guardaba aquellos registros correspondientes a casilleros del trayecto que no poseían acciones tácticas ni tiempo de espera, provocando que las rutas guardadas en los nuevos archivos se rompieran o perdieran tramos enteros de navegación.

Se rediseñó y amplió rigurosamente la condición del filtro dentro del doble bucle. Se incorporó la evaluación combinada de los punteros de salto (siguientex y siguientey dentro de los rangos válidos de la grilla) en conjunto con las coordenadas de posición, asegurando que ningún nodo activo del trayecto quede excluido al volcar la memoria RAM hacia el nuevo archivo binario en disco.

Operación 7: Problemas Detectados y Resolución

Debido a los arrastres de datos provocados por fallas previas en la carga en memoria y en los filtros de la Operación 6, el sistema sufría anomalías graves de representación: había misiones que no se visualizaban correctamente en el archivo de ataque, y en otros

casos extremos, las 40.000 casillas de la grilla se representaban como si estuvieran visitadas o activas, saturando el navegador con un bloque masivo de colores erróneos.

La solución integral abarcó tres frentes coordinados: se blindó la función de inicialización de la grilla para limpiar correctamente los estados iniciales, se corrigieron los filtros lógicos de persistencia en la Operación 6, y se ajustaron rigurosamente las condiciones if dentro del bucle de generación de celdas de la Operación 7. De este modo, se garantizó que únicamente los casilleros con datos de vuelo reales activen las clases visuales correspondientes, logrando un reporte web limpio, preciso y de alto rendimiento.

Registro de Herramientas Utilizadas

Nombre de la herramienta	**Dirección URL**	**Motivo y para qué se utilizó**	**Respuesta otorgada por la herramienta**	**Cambios o adaptaciones**
Gemini	https://gemini.google.com	Se utilizó para estructurar el trabajo práctico	La IA sugirió la utilización de archivos de cabecera .h y código fuente .cpp	
Gemini	https://gemini.google.com	Se utilizó para calcular las celdas aledañas	En este caso sugirió la utilización de la librería math para hacer la diferencia con valor absoluto	La adaptación fue (diferenciaX = abs(siguienteX - cx) y diferenciaY = abs(siguienteY - cy)), ademas de la validación (diferenciaX > 1 || diferenciaY > 1)
Gemini	https://gemini.google.com	Se utilizó para revisar una duda en la Operación 2, donde intentamos declararla	La IA indicó que se podía declarar como *void mostrarAtaque(Orden grilla[TAM_GRILLA][TAM_GRILLA])*, que era una forma válida de declararla, y que no era estrictamente	Se optó por la implementación de *Orden grilla[TAM_GRILLA][TAM_GRILLA])*, declarando

		como *void mostrarAtaque(Orden grilla[][TAM_GRILLA], int TAM_GRILLA)* o *void mostrarAtaque(Orden grilla[][TAM_GRILLA], int filas=TAM_GRILLA*	necesario pasar la cantidad de filas por separado	explícitamente la cantidad de filas y de columnas, para mantener la consistencia con la declaración del array en las demas funciones

Gemini	https://gemini.google.com	Se utilizó para los filtros de la operación 6 dentro del doble bucle for para tomar todos los registros válidos	La IA sugirió hacer:
			bool tieneAccion = (grilla[i][j].despegue ||
			
			grilla[i][j].aterrizaje ||
			
			grilla[i][j].soltarGranada1 ||
			
			grilla[i][j].soltarGranada2 ||
			
			grilla[i][j].ataqueKamikaze ||
			
			grilla[i][j].espera > 0);
			
			bool tieneSiguiente = (grilla[i][j].siguientex != -1 && grilla[i][j].siguientey != -1);
			
			if (tieneAccion || tieneSiguiente) { OrdenArchivo reg; reg.x = i; reg.y = j; reg.espera = grilla[i][j].espera; reg.soltarGranada1 = grilla[i][j].soltarGranada1; reg.soltarGranada2 = grilla[i][j].soltarGranada2;
				La adaptación fue:
				
				if (grilla[i][j].despegue==true||grilla[i][j].aterrizaje==true||
				
				grilla[i][j].soltarGranada1==true||
				grilla[i][j].soltarGranada2==true||
				
				grilla[i][j].ataqueKamikaze==true||
				grilla[i][j].espera > 0 ||
				(grilla[i][j].siguientex >= 0 &&
				grilla[i][j].siguientex < TAM_GRILLA &&

			reg.ataqueKamikaze = grilla[i][j].ataqueKamikaze; reg.aterrizaje = grilla[i][j].aterrizaje; reg.despegue = grilla[i][j].despegue;
			
			reg.siguientex = (grilla[i][j].siguientex == -1) ? 0 : grilla[i][j].siguientex; reg.siguientey = (grilla[i][j].siguientey == -1) ? 0 : grilla[i][j].siguientey;
				grilla[i][j].siguientey >= 0 && grilla[i][j].siguientey < TAM_GRILLA &&
				
				(grilla[i][j].siguientex != 0 || grilla[i][j].siguientey != 0 || i != 0 || j != 0)))

Gemini	https://gemini.google.com	Se utilizó para la personalización del html, además se planeó la utilización de un botón que haga zoom hacia donde está la ruta del drone, pero esto último fue descartado.	La IA sugirió el html embebido con el css, cuando solo se solicitó el css. fprintf(f, "<!DOCTYPE html>\n<html lang=\"es\">\n<head>\n"); fprintf(f, " <meta charset=\"UTF-8\">\n"); fprintf(f, " <title>Misión del Dron</title>\n"); fprintf(f, " <style>\n"); fprintf(f, " body { font-family: 'Segoe UI', Arial, sans-serif; background-color: #0f172a; color: #f8fafc; margin: 20px; }\n"); fprintf(f, " h1 { text-align: center; color: #38bdf8; font-size: 26px; margin-bottom: 20px; }\n"); fprintf(f, " .layout { display: flex; justify-content: center; gap: 20px; align-items: flex-start; }\n"); fprintf(f, " .table-container { max-width: 80vw;	La adaptación fue crear la constante y apartar el css para que no este embebido en el html const char CSS_ESTILOS[] = R"(<style> body { font-family: 'Segoe UI', Arial, sans-serif; background-color: #0f172a; color: #f8fafc; margin: 20px; } h1 { text-align: center; color: #38bdf8; font-size: 26px; margin-bottom: 20px; } .layout {

			max-height: 80vh; overflow: auto; border: 2px solid #334155; border-radius: 8px; background-color: #1e293b; padding: 10px; }\n"); fprintf(f, " table { border-collapse: collapse; }\n"); fprintf(f, " td { width: 14px; height: 14px; text-align: center; font-size: 10px; border: 1px solid #334155; color: #475569; }\n"); // Clases CSS por tipo de instrucción en la celda fprintf(f, " .c-despegue { background-color: #22c55e; color: #ffffff; font-weight: bold; } /* Verde */\n"); fprintf(f, " .c-kamikaze { background-color: #ef4444; color: #ffffff; font-weight: bold; } /* Rojo */\n"); fprintf(f, " .c-granada { background-color: #f59e0b; color: #ffffff; font-weight: bold; } /* Naranja */\n"); fprintf(f, " .c-espera { background-color: #06b6d4; color: #ffffff; font-weight: bold; } /* Cyan */\n"); fprintf(f, " .c-paso { background-color: #3b82f6; color: #ffffff; } /* Azul */\n"); // Estilos del panel de referencias lateral (Sidebar) fprintf(f, " .sidebar { background-color: #1e293b; border: 1px solid #334155; border-radius: 8px; padding: 15px; width: 220px; }\n"); fprintf(f, " .sidebar h3 { color:	display: flex; justify-content: center; gap: 20px; align-items: flex-start; } .table-container { max-width: 80vw; max-height: 80vh; overflow: auto; border: 2px solid #334155; border-radius: 8px; background-color: #1e293b; padding: 10px; } table { border-collapse: collapse; } td { width: 14px; height: 14px; text-align: center; font-size: 10px; border: 1px solid #334155; color: #475569; } /* Estilos tácticos por tipo de orden en la celda */ .c-despegue { background-color: #22c55e; color: #ffffff; font-weight: bold; } /* Verde */ .c-kamikaze {

			#38bdf8; margin-top: 0; font-size: 16px; border-bottom: 1px solid #334155; padding-bottom: 8px; }\n"); fprintf(f, " .item { display: flex; align-items: center; margin-bottom: 10px; font-size: 13px; }\n"); fprintf(f, " .box { width: 18px; height: 18px; text-align: center; line-height: 18px; font-weight: bold; border-radius: 3px; margin-right: 10px; font-size: 11px; }\n"); fprintf(f, " </style>\n</head>\n<body>\n"); fprintf(f, " Visualización del Recorrido del Dron \n"); fprintf(f, " <div class=\"layout\">\n"); fprintf(f, " <div class=\"table-container\">\n"); fprintf(f, " <table>\n"); fprintf(f, " <div class=\"sidebar\">\n"); fprintf(f, " Referencias \n"); fprintf(f, " <div class=\"item\"><div class=\"box c-despegue\">D Despegue \n"); fprintf(f, " <div class=\"item\"><div class=\"box c-kamikaze\">F Aterrizaje / Kamikaze \n"); fprintf(f, " <div class=\"item\"><div class=\"box	background-color: #ef4444; color: #ffffff; font-weight: bold; } /* Rojo */ .c-granada { background-color: #f59e0b; color: #ffffff; font-weight: bold; } /* Naranja */ .c-espera { background-color: #06b6d4; color: #ffffff; font-weight: bold; } /* Cyan */ .c-paso { background-color: #3b82f6; color: #ffffff; } /* Azul */ /* Leyenda lateral */ .sidebar { background-color: #1e293b; border: 1px solid #334155; border-radius: 8px; padding: 15px; width: 220px; } .sidebar h3 { color: #38bdf8; margin-top: 0; font-size: 16px; border-bottom: 1px solid #334155; padding-bottom: 8px; } .item

			c-granada\">G Soltar Granada \n"); fprintf(f, " <div class=\"item\"><div class=\"box c-espera\">E Tiempo de Espera \n"); fprintf(f, " <div class=\"item\"><div class=\"box c-paso\">• Movimiento de Ruta \n"); fprintf(f, " \n"); fprintf(f, " \n"); fprintf(f, "</body>\n</html>\n");	{ display: flex; align-items: center; margin-bottom : 10px; font-size: 13px; } .box { width: 18px; height: 18px; text-align: center; line-height: 18px; font-weight: bold; border-radius: 3px; margin-right: 10px; font-size: 11px; } </style>)";
Gemini	https://gemini.google.com	Se lo utilizó para estructurar y corregir errores de redacción y continuidad del informe	La IA sugirió vocabulario más formal, conectores académicos, planteamientos extensos y la utilización de un índice donde se detallaba por la funciones que actuaban en memoria y las que no	Tomamos las sugerencias de estructuración del índice y lo dividimos por operaciones con su respectiva hipótesis, solución, casos de prueba y por último los problemas generales